Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



Programación en Móviles Avanzado

LABORATORIO N° 11

Core Data - SwiftData

DOCENTE:

JUAN LEÓN

CURSO:

Programación en Móviles Avanzado
5 C24

LABORATORIO 11 · SEMANA 11

Manejo de datos con Core Data y SwiftData

Crear, conectar y consultar datos locales con UIKit y con SwiftUI

I. Capacidades

- Implementar persistencia de datos mediante operaciones CRUD (crear, leer, actualizar y eliminar).
- Explicar dónde se guardan los datos de una app y cómo se conecta la pantalla con ellos.
- Diseñar un modelo de datos con entidades, atributos y una relación de uno a muchos.
- Consultar datos con ordenamiento y filtros (fetch requests y @Query).

II. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

III. Fundamento teórico

- Revise el material de la semana correspondiente antes del desarrollo del laboratorio.
- Repase UITableView y UIAlertController (laboratorios 4 a 6) y List, @State y NavigationStack (laboratorio 8).

IV. Normas empleadas

- No aplica.

V. Recursos

- Cada pareja trabaja con una computadora con Xcode y el simulador de iPhone.
- Una carpeta de código de apoyo con todos los archivos de este laboratorio, para consultar si algo no compila.

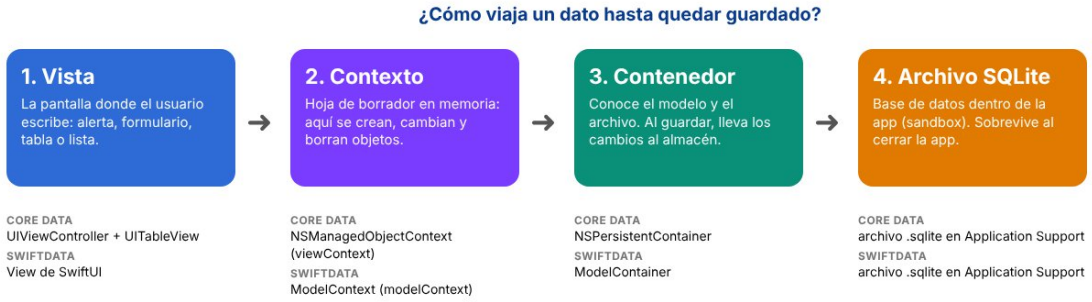
VI. Metodología para el desarrollo de la tarea

- Al terminar cada parte hacen **commit y push** a su repositorio, con un mensaje claro.
- El trabajo de la semana se entrega en la próxima clase y se defiende oralmente.

VI. Procedimiento

Parte 0 · La idea (15 minutos)

Una app que solo guarda datos en variables los pierde al cerrarse. **Persistir** es guardar los datos en un archivo dentro del propio teléfono, para recuperarlos después. Core Data y SwiftData hacen ese trabajo por nosotros: no escribimos SQL, trabajamos con objetos de Swift.



Equivalencias entre los dos frameworks

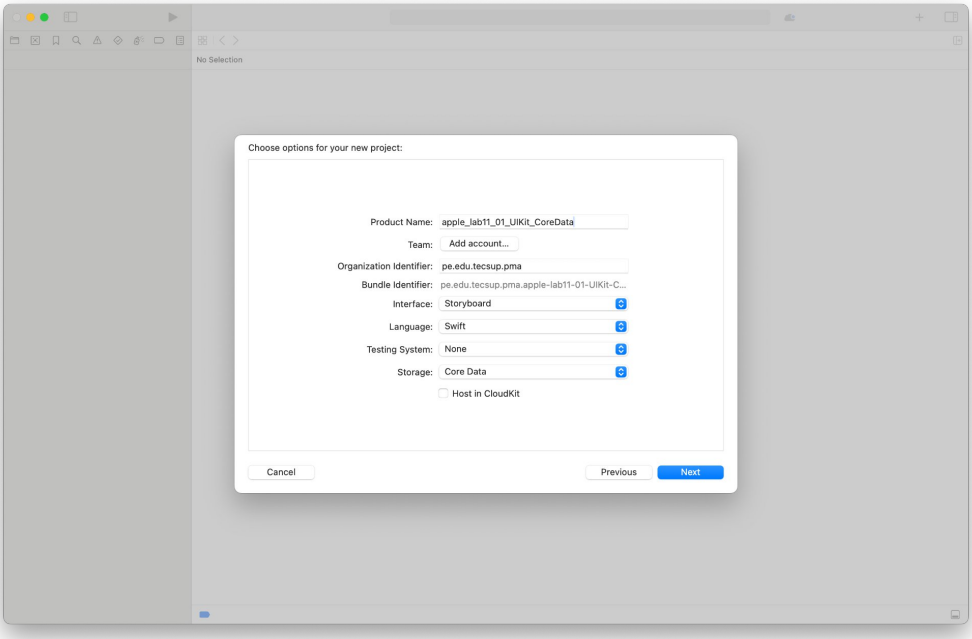
Concepto	Core Data (UIKit)	SwiftData (SwiftUI)
Modelo de datos	Archivo .xcdatamodeld con editor visual	Clases con la anotación @Model
Entidad y atributo	Entity y Attribute en el editor	Clase y propiedades var
Un registro	Objeto de la clase generada (Contacto)	Instancia de la clase (Alumno)
Contexto	NSManagedObjectContext (viewContext)	ModelContext (modelContext)
Contenedor	NSPersistentContainer	ModelContainer
Crear	NSEntityDescription.insertNewObject	modelContext.insert(objeto)
Leer	NSFetchRequest, NSSortDescriptor, NSPredicate	@Query, filtro y ordenamiento
Guardar	contexto.save()	Guardado automático; save() lo asegura
Borrar	contexto.delete(objeto)	modelContext.delete(objeto)
Relación	Relationship en el editor (To One / To Many)	@Relationship(deleteRule:inverse:)
Actualizar la pantalla	Manual: reloadData()	Automática: la vista se redibuja sola

Parte 1 · Actividad 01: Core Data con UIKit (Contacto)

Construiremos una agenda de contactos con una tabla, y recorreremos las cuatro operaciones del CRUD.

Paso 1 · Crear el proyecto

1. En Xcode: File ▸ New ▸ Project ▸ iOS ▸ App.
2. Product Name: apple_lab11_01_UIKit_CoreData. Interface: **Storyboard**. Language: **Swift**. Storage: **Core Data**.



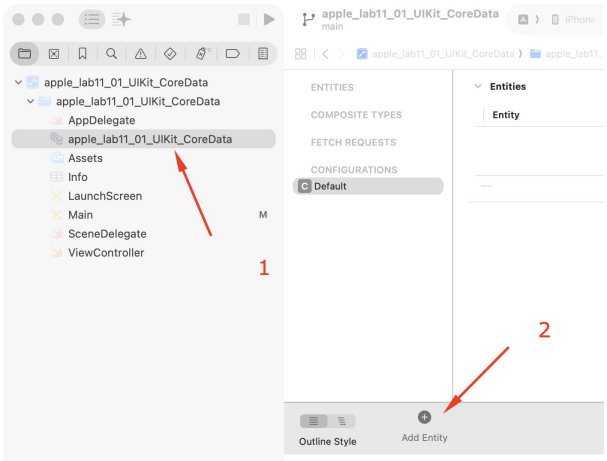
Creación del proyecto con Storage: Core Data

Qué pasó por dentro

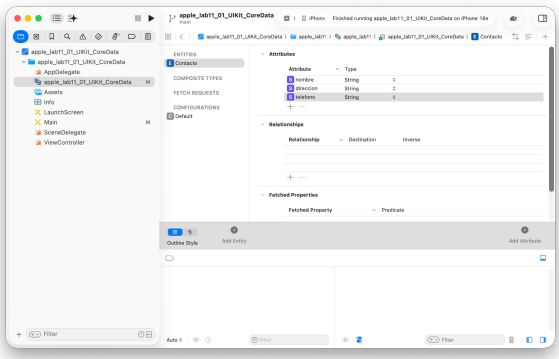
Al elegir Core Data, Xcode agregó dos cosas: el archivo del modelo (.xcdatamodeld) y, en AppDelegate, una propiedad llamada persistentContainer. Ya tenemos la infraestructura de datos; falta definir qué datos guardar.

Paso 2 · Definir el modelo de datos

1. Abre el archivo con extensión .xcdatamodeld (tiene el nombre del proyecto).
2. Pulsa **Add Entity** (abajo) y renombra la entidad como **Contacto**.
3. Con la entidad seleccionada, pulsa **Add Attribute** tres veces: **nombre**, **telefono** y **direccion**, los tres de tipo **String**.



1. Abrir el modelo · 2. Add Entity



Entidad Contacto con sus tres atributos

Qué pasó por dentro

Una **entidad** es como una tabla; cada **atributo**, una columna. Xcode genera por ti la clase Contacto (Codegen: Class Definition), que usaremos en el código como cualquier clase de Swift. Los atributos quedan como opcionales (String?), por eso el código usa ?? "".

Paso 3 · Leer cómo se conecta Core Data

Abre AppDelegate.swift y localiza este fragmento que Xcode generó. **No hay que escribirlo**, solo entenderlo:

```
// Fragmento que Xcode genera en AppDelegate.swift al elegir "Core Data".
// No hay que escribirlo: sólo leerlo y entenderlo.
lazy var persistentContainer: NSPersistentContainer = {
    // El nombre debe coincidir con el archivo .xcdatamodeld
    let container = NSPersistentContainer(name: "apple_lab11_01_UIKit_CoreData")
    container.loadPersistentStores(completionHandler: { (storeDescription, error) in
        if let error = error as NSError? {
            fatalError("Unresolved error \(error), \(error.userInfo)")
        }
    })
    return container
}()
```

> **Qué pasó por dentro**
> El nombre del contenedor debe coincidir con el del archivo .xcdatamodeld. loadPersistentStores abre el archivo SQLite de la app, o lo crea si no existe. Desde ahora, cualquier parte de la app puede pedir el contexto con persistentContainer.viewContext.

Paso 4 · Diseñar la pantalla

1. En Main.storyboard, selecciona el View Controller y usa Editor ▸ Embed In ▸ **Navigation Controller**.
2. Agrega un **Table View** que ocupe toda la pantalla y fíjalo a los bordes con constraints.
3. Selecciona la celda prototipo: Style **Subtitle** e Identifier **Cell**.
4. Agrega un **Bar Button Item** a la derecha de la barra y cambia su System Item a **Add** (+). Pon el título de la pantalla "Lista de Contactos - Tecsup".



Storyboard: Navigation Controller y pantalla con Table View

5. Conecta con Control+arrastre hacia ViewController.swift: un **Outlet** para la tabla llamado TablaContacto y una **Action** para el botón + llamada agregarContacto (Type: UIBarButtonItem).

Paso 5 · Código base: contexto y ruta del archivo

Reemplaza el contenido de ViewController.swift por la estructura base. Añade los métodos de los siguientes pasos **dentro de la clase**, uno por uno.

Archivo: **ViewController.swift**

```
import UIKit
import CoreData

class ViewController: UIViewController, UITableViewDelegate, UITableViewDataSource {

    // Outlet conectado al UITableView del storyboard
    @IBOutlet weak var TablaContacto: UITableView!

    // Arreglo que almacena los contactos obtenidos desde Core Data
    var contactos = [Contacto]()

    // Método llamado al cargar la vista
    override func viewDidLoad() {
        super.viewDidLoad()
        TablaContacto.dataSource = self
        TablaContacto.delegate = self
        mostrarRutaBaseDeDatos()
        cargarInfoCoreData()   // Carga los datos almacenados en Core Data
    }
```

Después agrega la conexión con Core Data:

```
    // MARK: - Conexión con Core Data

    // Obtiene el contexto de Core Data para realizar operaciones de lectura y escritura
    func conexion() -> NSManagedObjectContext {
        let delegate = UIApplication.shared.delegate as! AppDelegate
        return delegate.persistentContainer.viewContext
    }

    // Imprime en consola dónde está el archivo SQLite de la app
    func mostrarRutaBaseDeDatos() {
        let delegate = UIApplication.shared.delegate as! AppDelegate
        let ruta = delegate.persistentContainer.persistentStoreDescriptions.first?.url?.path ??
"sin ruta"
        print("Base de datos SQLite en: \(ruta)")
    }
```

Qué pasó por dentro

conexion() obtiene el contexto del AppDelegate. Cada operación del CRUD empieza pidiendo este contexto. mostrarRutaBaseDeDatos() imprime en la consola de Xcode dónde quedó el archivo SQLite: lo usaremos al comprobar la persistencia.

Paso 6 · READ: leer los datos y mostrarlos

Predice antes de ejecutar

Todavía no hemos guardado nada. ¿Qué verás en la tabla al ejecutar? ¿Qué texto aparecerá en la consola?

Archivo: **ViewController.swift (dentro de la clase)**

```
    // MARK: - READ: leer los datos

    // Carga los contactos almacenados y actualiza el arreglo local
    func cargarInfoCoreData() {
        let contexto = conexion()
        let fetchRequest: NSFetchRequest<Contacto> = Contacto.fetchRequest()
        // Orden alfabético por nombre
        fetchRequest.sortDescriptors = [NSSortDescriptor(key: "nombre", ascending: true)]
        do {
            contactos = try contexto.fetch(fetchRequest)
        } catch {
            print("Error al cargar la BD: \(error)")
        }
    }
```

```
    // MARK: - UITableViewDataSource

    // Número de filas en la tabla
    func tableView(_ tableView: UITableView, numberOfRowsInSection section: Int) -> Int {
        return contactos.count
    }

    // Configura cada celda de la tabla
    func tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath) -> UITableViewCell
{
        let celda = TablaContacto.dequeueReusableCell(withIdentifier: "Cell", for: indexPath)
        celda.textLabel?.text = contactos[indexPath.row].nombre
        celda.detailTextLabel?.text = "\(contactos[indexPath.row].telefono ?? "") |
\(contactos[indexPath.row].direccion ?? "")"
        return celda
    }
```

Qué esperas ver

Al ejecutar (▶) en el simulador: una tabla **vacía** y, en la consola de Xcode, una línea "Base de datos SQLite en: ..." con la ruta del archivo.

Qué pasó por dentro

NSFetchRequest es la consulta: "tráeme todos los Contacto". Con sortDescriptors los pedimos ordenados por nombre. Los resultados quedan en el arreglo contactos, y la tabla se dibuja a partir de ese arreglo. La tabla nunca habla con la base de datos directamente.

Paso 7 · CREATE: agregar contactos

Predice antes de ejecutar

Después de guardar un contacto, ¿se verá en la tabla sin hacer nada más? ¿Qué línea crees que hace que aparezca?

Archivo: **`ViewController.swift (dentro de la clase)`**

```
    // MARK: - CREATE: agregar un contacto

    // Acción vinculada al Bar Button Item (+)
    @IBAction func agregarContacto(_ sender: UIBarButtonItem) {
        let alert = UIAlertController(title: "Agregar Contacto", message: "Nuevo Contacto",
preferredStyle: .alert)
        alert.addTextField { $0.placeholder = "Nombre" }
        alert.addTextField { $0.placeholder = "Telefono" }
        alert.addTextField { $0.placeholder = "Direccion" }

        let actionAceptar = UIAlertAction(title: "Aceptar", style: .default) { _ in
            guard let nombre = alert.textFields?[0].text, !nombre.isEmpty else { return }
            let telefono = alert.textFields?[1].text ?? ""
            let direccion = alert.textFields?[2].text ?? ""

            let contexto = self.conexion()
            // Crea un nuevo objeto Contacto en el contexto (todavía NO está guardado)
            let entidad = NSEntityDescription.insertNewObject(forEntityName: "Contacto", into:
contexto) as! Contacto
            entidad.nombre = nombre
            entidad.telefono = telefono
            entidad.direccion = direccion

            do {
                try contexto.save()          // Aquí se escribe en el archivo SQLite
                self.cargarInfoCoreData()    // Vuelve a leer los datos
                self.TablaContacto.reloadData()
            } catch {
                print("Error al guardar el dato: \(error)")
            }
        }
        alert.addAction(actionAceptar)
        alert.addAction(UIAlertAction(title: "Cancelar", style: .cancel))
```

```
        present(alert, animated: true)
    }
```



Alerta para agregar



Tabla con contactos

Qué esperas ver

Pulsa + , completa nombre, teléfono y dirección y pulsa Aceptar: el contacto aparece en la tabla. Agrega al menos tres.

Qué pasó por dentro

insertNewObject crea el objeto **solo en el contexto** (borrador). contexto.save() lo escribe en el archivo SQLite. Luego volvemos a leer (cargarInfoCoreData) y le pedimos a la tabla que se redibuje (reloadData): UIKit no se actualiza solo.

Paso 8 · Comprobar que los datos persisten

1. Detén la app (■) y vuelve a ejecutarla (▶).
2. Los contactos siguen ahí. Eso es persistencia: vienen del archivo, no de la memoria.
3. Opcional: copia la ruta impresa en la consola y abre Terminal. Ejecuta: sqlite3 "RUTA" "SELECT ZNOMBRE, ZTELEFONO FROM ZCONTACTO;" y verás tus datos como filas de una tabla.

Paso 9 · UPDATE: editar un contacto

Archivo: **`ViewController.swift (dentro de la clase)`**

```
    // MARK: - UPDATE: editar un contacto (al tocar una fila)

    func tableView(_ tableView: UITableView, didSelectRowAt indexPath: IndexPath) {
        tableView.deselectRow(at: indexPath, animated: true)
        let contacto = contactos[indexPath.row]

        let alert = UIAlertController(title: "Editar Contacto", message: "Modifica los datos",
preferredStyle: .alert)
        alert.addTextField { $0.text = contacto.nombre }
        alert.addTextField { $0.text = contacto.telefono }
        alert.addTextField { $0.text = contacto.direccion }

        let actionGuardar = UIAlertAction(title: "Guardar", style: .default) { _ in
            guard let nombre = alert.textFields?[0].text, !nombre.isEmpty else { return }
            // Cambiar las propiedades del objeto y guardar el contexto
            contacto.nombre = nombre
            contacto.telefono = alert.textFields?[1].text
            contacto.direccion = alert.textFields?[2].text
            do {
```

```
                try self.conexion().save()
                self.cargarInfoCoreData()
                self.TablaContacto.reloadData()
            } catch {
                print("Error al actualizar: \(error)")
            }
        }
        alert.addAction(actionGuardar)
        alert.addAction(UIAlertAction(title: "Cancelar", style: .cancel))
        present(alert, animated: true)
    }
```

Qué esperas ver

Al tocar una fila aparece una alerta con los datos actuales. Cambia uno, pulsa Guardar y la tabla muestra el cambio. Detén y ejecuta de nuevo: el cambio se mantiene.

Qué pasó por dentro

Para actualizar no hay un método especial: basta cambiar las propiedades del objeto y guardar el contexto. Por eso este paso casi no se parece a "insertar".

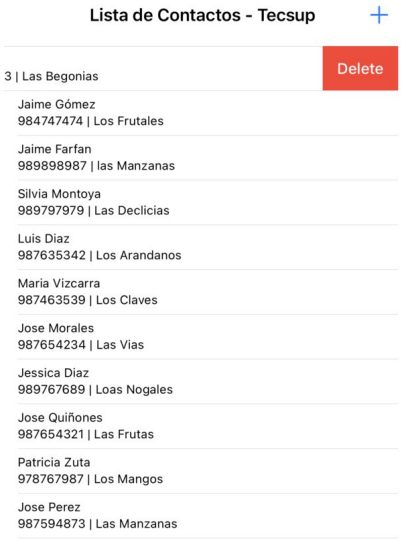
Paso 10 · DELETE: eliminar un contacto

Archivo: **`ViewController.swift (dentro de la clase)`**

```
    // MARK: - DELETE: eliminar un contacto

    // Elimina un contacto específico dado el índice en la tabla
    func eliminarContacto(at indexPath: IndexPath) {
        let contexto = conexion()
        let contactoEliminar = contactos[indexPath.row]
        contexto.delete(contactoEliminar)       // Marca el objeto para borrarlo
        do {
            try contexto.save()                  // Confirma el borrado en SQLite
            contactos.remove(at: indexPath.row)  // Lo quita del arreglo local
            TablaContacto.deleteRows(at: [indexPath], with: .automatic)
        } catch {
            print("Error al eliminar: \(error)")
        }
    }

    // Permite eliminar deslizando la fila hacia la izquierda
    func tableView(_ tableView: UITableView, commit editingStyle: UITableViewCell.EditingStyle,
                   forRowAt indexPath: IndexPath) {
        if editingStyle == .delete {
            eliminarContacto(at: indexPath)
        }
    }
```



Deslizar una fila hacia la izquierda muestra Delete

Qué esperas ver

Al deslizar una fila a la izquierda aparece Delete. Al pulsarlo la fila desaparece con animación y no vuelve al reiniciar la app.

Ojo

Si el botón Delete no aparece, revisa que la tabla tenga dataSource y delegate (los asignamos en viewDidLoad) y que el método commit editingStyle esté dentro de la clase.

Commit y push: "Lab11 parte 1: CRUD de contactos con Core Data".

Paso 11 · Fetch requests: filtrar y buscar

Una consulta puede incluir un **predicate** (filtro). Crea un archivo nuevo llamado ViewController+Relacion.swift. Primero escribe solo las dos funciones de búsqueda:

```
import UIKit
import CoreData

extension ViewController {

    // fetch request con predicate simple: buscar por texto en el nombre
    func buscarContactos(texto: String) -> [Contacto] {
        let request: NSFetchRequest<Contacto> = Contacto.fetchRequest()
        request.predicate = NSPredicate(format: "nombre CONTAINS[cd] %@", texto)
        request.sortDescriptors = [NSSortDescriptor(key: "nombre", ascending: true)]
        return (try? conexion().fetch(request)) ?? []
    }
}
```

Para probarla, agrega temporalmente en viewDidLoad, después de cargarInfoCoreData():

```
contactos = buscarContactos(texto: "juan")   // PRUEBA: borra esta línea después
TablaContacto.reloadData()
```

Qué esperas ver

La tabla muestra solo los contactos cuyo nombre contiene "juan", sin importar mayúsculas ni tildes ([cd]). Borra la línea de prueba al terminar.

Paso 12 · Relación de uno a muchos: Grupo y Contacto

Un **grupo** tiene muchos contactos, y cada contacto pertenece a un grupo.

1. En el editor del modelo agrega una entidad **Grupo** con un atributo **nombre** (String).
2. En Contacto, pulsa + en Relationships: nombre **grupo**, Destination **Grupo**, Type **To One**.
3. En Grupo, agrega la relación **contactos**: Destination **Contacto**, Type **To Many**, Inverse **grupo**. Delete Rule: **Nullify**.
4. Comprueba en Contacto que la relación grupo tiene Inverse = contactos.

Agrega al archivo ViewController+Relacion.swift, dentro de la extensión:

```
    // Crea un grupo y se lo asigna al primer contacto. Ejecútala UNA sola vez
    // (por ejemplo desde viewDidLoad) y luego vuelve a comentarla.
    func probarRelacion() {
        let contexto = conexion()
        let grupo = NSEntityDescription.insertNewObject(forEntityName: "Grupo", into: contexto)
as! Grupo
        grupo.nombre = "Amigos"
        if let primero = contactos.first {
            primero.grupo = grupo            // El lado "uno" de la relación
        }
        do {
            try contexto.save()
            print("El grupo \(grupo.nombre ?? "") tiene \(grupo.contactos?.count ?? 0)
contacto(s)")
        } catch {
            print("Error al guardar la relación: \(error)")
        }
    }

    // fetch request con predicate que recorre la relación
    func contactosDelGrupo(_ nombreGrupo: String) -> [Contacto] {
        let request: NSFetchRequest<Contacto> = Contacto.fetchRequest()
        request.predicate = NSPredicate(format: "grupo.nombre == %@", nombreGrupo)
        return (try? conexion().fetch(request)) ?? []
    }
```

Qué esperas ver

Al llamar probarRelacion() una vez desde viewDidLoad, la consola imprime "El grupo Amigos tiene 1 contacto(s)". Después comenta la llamada para no crear grupos repetidos.

Qué pasó por dentro

Basta asignar contacto.grupo = grupo para enlazarlos; Core Data completa el lado inverso (grupo.contactos) automáticamente. El predicate "grupo.nombre == %@" recorre la relación como si fuera una propiedad.

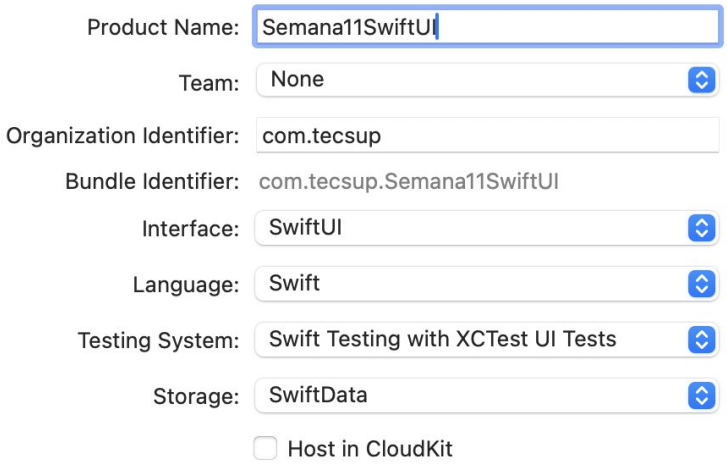
Commit y push: "Lab11 parte 1: filtro y relación en Core Data".

Parte 2 · Actividad 02: SwiftData con SwiftUI (Alumno)

Ahora resolvemos el mismo problema con otra tecnología. Fíjate en que **no hay editor de modelo, ni AppDelegate, ni reloadData()**: el modelo es una clase de Swift y la pantalla se actualiza sola.

Paso 1 · Crear el proyecto

1. File ▸ New ▸ Project ▸ iOS ▸ App. Product Name: apple_lab11_02_SwiftUI_SwiftData.
2. Interface: **SwiftUI**. Language: **Swift**. Storage: **SwiftData**.



Proyecto SwiftUI con Storage: SwiftData

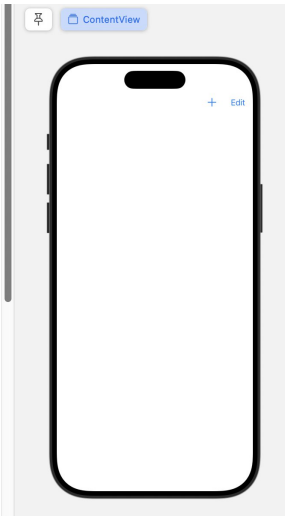
Paso 2 · Qué trae la plantilla

Xcode crea un ejemplo con una clase **Item** y un ContentView que lista fechas. Sirve para ver las piezas, pero la reemplazaremos por nuestro modelo.

```swift
//  Created by Juan Leon -Jaime Gómez
import SwiftUI
import SwiftData

struct ContentView: View {
    @Environment(\.modelContext) private var modelContext
    @Query private var items: [Item]

    var body: some View {
        NavigationSplitView {
            List {
                ForEach(items) { item in
                    NavigationLink {
                        Text("Item at \(item.timestamp, format: Date.FormatStyle(date: .numeric, time: .standard))")
                    } label: {
                        Text(item.timestamp, format: Date.FormatStyle(date: .numeric, time: .standard))
                    }
                }
                .onDelete(perform: deleteItems)
            }
            .toolbar {
                ToolbarItem(placement: .navigationBarTrailing) {
                    EditButton()
                }
                ToolbarItem {
                    Button(action: addItem) {
                        Label("Add Item", systemImage: "plus")
                    }
                }
            }
        } detail: {
            Text("Select an item")
        }
    }
```

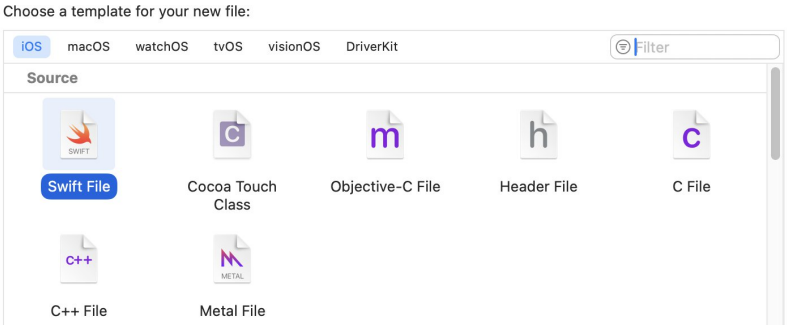


Plantilla por defecto con Item

> **Ojo**
> No borres Item.swift todavía: el ContentView de la plantilla lo usa y el proyecto dejaría de compilar. Lo borraremos en el paso 7, cuando ya hayamos reemplazado ContentView.

Paso 3 · Crear el modelo de datos: Alumno

1. File ► New ► File ► **Swift File** y nómbralo **Alumno**.
2. Reemplaza su contenido por:



Nuevo archivo Swift

Archivo: **`Alumno.swift`**

```
import Foundation
// Importamos SwiftData para poder usar la anotación @Model y la persistencia automática
import SwiftData

// Indicamos que esta clase representa una entidad persistente en la base de datos
@Model
class Alumno {
    var idAlumno: UUID        // Identificador único del alumno
    var apellido: String
    var nombre: String
    var dni: String

    // Constructor que inicializa un nuevo objeto Alumno con los datos proporcionados
    init(idAlumno: UUID = UUID(), apellido: String, nombre: String, dni: String) {
        self.idAlumno = idAlumno
        self.apellido = apellido
        self.nombre = nombre
        self.dni = dni
    }
}
```

> **Qué pasó por dentro**
> @Model convierte una clase normal en una **entidad persistente**: cada propiedad var será una columna. Es el equivalente del editor .xcdatamodeld de la Parte 1, pero escrito en código.

Paso 4 · Dónde se guarda: ModelContainerProvider

Nuevo archivo Swift llamado **ModelContainerProvider**:

Archivo: **`ModelContainerProvider.swift`**

```
import Foundation
import SwiftData

@MainActor // Garantiza que esta clase se ejecute en el hilo principal (UI-safe)
class ModelContainerProvider {

    // Patrón Singleton: compartimos una única instancia del contenedor en toda la app
    static let shared = ModelContainerProvider()

    let container: ModelContainer // Contenedor principal de SwiftData

    private init() {
        // Esquema: las clases modelo que queremos persistir
        let schema = Schema([Alumno.self])

        do {
            // Dónde y cómo se guardará la base de datos SQLite
            let config = ModelConfiguration(
                "AlumnosData",
                schema: schema,
                url: FileManager.default
                    .urls(for: .applicationSupportDirectory, in: .userDomainMask)
                    .first!
                    .appendingPathComponent("AlumnosData.sqlite")
            )
```

```
            // Para ver dónde quedó el archivo
            print("Base de datos SQLite en: \(config.url.path)")

            container = try ModelContainer(for: schema, configurations: [config])
        } catch {
            fatalError("Error al inicializar ModelContainer: \(error)")
        }
    }
}
```

> **Qué pasó por dentro**
> El **Schema** lista las clases que se guardan. El **ModelConfiguration** dice en qué archivo SQLite. El **ModelContainer** une ambos y es el equivalente del persistentContainer de AppDelegate. Lo hacemos singleton (static let shared) para tener uno solo en toda la app.

Paso 5 · Conectar el contenedor a la app

Abre el archivo principal (el que termina en App y tiene @main) y déjalo así. Cambia el nombre de la estructura por el de tu proyecto si es distinto:

Archivo: **apple_lab11_02_SwiftUI_SwiftDataApp.swift**

```
import SwiftUI
import SwiftData

@main // Punto de entrada principal de la aplicación
struct AlumnosApp: App {
    var body: some Scene {
        WindowGroup {
            ContentView() // Vista inicial: la lista de alumnos
                // Contenedor persistente para toda la app:
                // los datos se guardan en disco y no se borran al cerrar la app
                .modelContainer(ModelContainerProvider.shared.container)
        }
    }
}
```

> **Qué pasó por dentro**
> .modelContainer(...) pone el contenedor en el **entorno** de SwiftUI: cualquier vista descendiente puede pedir el contexto con @Environment(\.modelContext) y consultar con @Query, sin pasarlos a mano.

Paso 6 · Formulario: NuevoAlumnoView (CREATE)

Archivo: **NuevoAlumnoView.swift (nuevo archivo SwiftUI View)**

```
import SwiftUI
import SwiftData

struct NuevoAlumnoView: View {

    @Environment(\.modelContext) private var modelContext // Contexto del modelo (insertar,
guardar, eliminar)
    @Environment(\.dismiss) private var dismiss           // Cierra esta vista (como un "volver")

    @State private var apellido = ""
    @State private var nombre = ""
    @State private var dni = ""

    var body: some View {
        NavigationStack {
            Form {
                Section(header: Text("Datos del Alumno")) {
                    TextField("Apellido", text: $apellido)
                    TextField("Nombre", text: $nombre)
                    TextField("DNI", text: $dni)
                        .keyboardType(.numberPad)
                }
```

```
                Button("Guardar Alumno") {
                    // CREATE: crear el objeto e insertarlo en el contexto
                    let nuevoAlumno = Alumno(apellido: apellido, nombre: nombre, dni: dni)
                    modelContext.insert(nuevoAlumno)

                    // Guardamos de inmediato para asegurar la persistencia
                    do {
                        try modelContext.save()
                    } catch {
                        print("Error al guardar alumno: \(error)")
                    }
                    dismiss() // Cierra el formulario
                }
                .disabled(apellido.isEmpty || nombre.isEmpty || dni.isEmpty) // Desactiva si hay
campos vacíos
            }
            .navigationTitle("Nuevo Alumno")
            .toolbar {
                ToolbarItem(placement: .cancellationAction) {
                    Button("Cancelar") {
                        dismiss() // Cierra el formulario sin guardar
                    }
                }
            }
        }
    }
}
```

Predice antes de ejecutar

El botón Guardar tiene .disabled(...). ¿Cuándo estará inactivo? ¿Qué línea crea el registro y cuál lo escribe en el archivo?

Paso 7 · Pantalla principal: ContentView (READ y DELETE)

Reemplaza todo el contenido de ContentView.swift por la versión inicial. Después **borra Item.swift** (Delete ▸ Move to Trash).

Archivo: **`ContentView.swift (versión 1)`**

```
import SwiftUI
import SwiftData // Para consultar y manipular datos persistentes

struct ContentView: View {
    @Environment(\.modelContext) private var modelContext // Contexto: insertar, borrar, guardar

    // READ: consulta automática de todos los alumnos ordenados por apellido
    @Query(sort: \Alumno.apellido) private var alumnos: [Alumno]

    @State private var alumnoSeleccionado: Alumno?   // Alumno seleccionado en la lista
    @State private var mostrarFormulario = false     // Muestra el formulario de nuevo alumno

    var body: some View {
        NavigationSplitView { // Lista a la izquierda, detalle a la derecha
            List(selection: $alumnoSeleccionado) {
                ForEach(alumnos) { alumno in
                    NavigationLink(value: alumno) {
                        VStack(alignment: .leading) {
                            Text("\(alumno.apellido), \(alumno.nombre)")
                                .font(.headline)
                            Text("DNI: \(alumno.dni)")
                                .font(.subheadline)
                        }
                    }
                }
                .onDelete(perform: deleteAlumnos) // Eliminar con swipe o modo edición
            }
            .navigationTitle(" Alumnos")
            .toolbar {
                ToolbarItem(placement: .navigationBarTrailing) {
                    EditButton()
                }
                ToolbarItem {
                    Button {
                        mostrarFormulario = true
                    } label: {
```

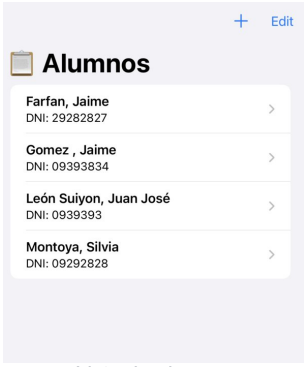
```
                        Label("Nuevo Alumno", systemImage: "plus")
                    }
                }
            }
        } detail: {
            if let alumno = alumnoSeleccionado {
                VStack(alignment: .leading, spacing: 10) {
                    Text("□ Detalles del Alumno")
                        .font(.title2)
                    Text("Nombre: \(alumno.nombre)")
                    Text("Apellido: \(alumno.apellido)")
                    Text("DNI: \(alumno.dni)")
                }
                .padding()
            } else {
                Text("Selecciona un alumno")
            }
        }
        // Formulario de nuevo alumno como hoja flotante
        .sheet(isPresented: $mostrarFormulario) {
            NuevoAlumnoView()
        }
    }

    // DELETE: eliminar uno o más alumnos desde la lista
    private func deleteAlumnos(offsets: IndexSet) {
        withAnimation {
            for index in offsets {
                let alumno = alumnos[index]
                if alumnoSeleccionado == alumno { alumnoSeleccionado = nil }
                modelContext.delete(alumno)
            }
        }
    }
}

#Preview {
    ContentView()
        .modelContainer(for: [Alumno.self], inMemory: true)
}
```

Qué esperas ver

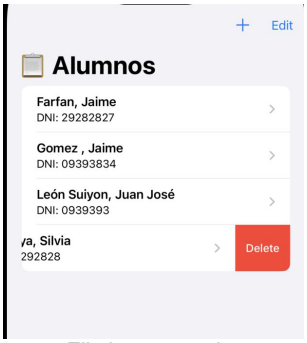
Al ejecutar: pantalla con "Selecciona un alumno" y lista vacía. Pulsa + , llena el formulario y Guarda: el alumno aparece **sin que escribas nada para refrescar**. Al seleccionarlo se ve el detalle. Desliza una fila a la izquierda para eliminar.



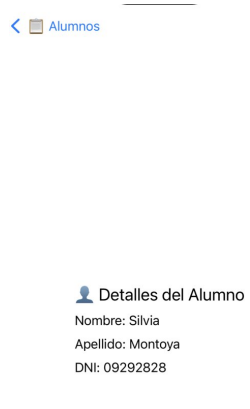
Lista de alumnos



Nuevo alumno



Eliminar con swipe



Detalle

Qué pasó por dentro

@Query(sort: ...) es el READ: SwiftData vigila la base y, cuando algo cambia (insert o delete), vuelve a dibujar la vista con la lista nueva. Esta es la gran diferencia con UIKit, donde nosotros llamábamos reloadData().

Paso 8 · Comprobar que los datos persisten

1. Detén la app y vuelve a ejecutarla: los alumnos siguen ahí.

2. Busca en la consola la línea "Base de datos SQLite en: ..." con la ruta del archivo AlumnosData.sqlite.
3. Comparación: en la Parte 1 el archivo se llamaba como el proyecto y las tablas empezaban con Z (ZCONTACTO). Aquí SwiftData usa la misma tecnología de fondo (SQLite).

Commit y push: "Lab11 parte 2: CRUD de alumnos con SwiftData".

Paso 9 · UPDATE: editar un alumno

Crea el archivo **EditarAlumnoView.swift**:

Archivo: **`EditarAlumnoView.swift`**

```
import SwiftUI
import SwiftData

// UPDATE: editar un alumno existente.
// @Bindable permite enlazar los campos directamente a las propiedades del objeto @Model.
// SwiftData detecta el cambio y lo guarda solo.
struct EditarAlumnoView: View {

    @Bindable var alumno: Alumno
    @Environment(\.dismiss) private var dismiss

    var body: some View {
        NavigationStack {
            Form {
                Section(header: Text("Datos del Alumno")) {
                    TextField("Apellido", text: $alumno.apellido)
                    TextField("Nombre", text: $alumno.nombre)
                    TextField("DNI", text: $alumno.dni)
                        .keyboardType(.numberPad)
                }
            }
            .navigationTitle("Editar Alumno")
            .toolbar {
                ToolbarItem(placement: .confirmationAction) {
                    Button("Listo") {
                        dismiss()
                    }
                }
            }
        }
    }
}
```

En **ContentView.swift** haz tres cambios:

1. Agrega esta variable junto a las otras @State:

```
    @State private var alumnoAEditar: Alumno?        // Alumno que se está editando
```

2. En el detalle, después del Text("DNI: ..."), agrega el botón:

```
                    Button("Editar") {
                        alumnoAEditar = alumno
                    }
```

3. Después del primer .sheet(...) agrega la hoja de edición:

```
        // Formulario de edición
        .sheet(item: $alumnoAEditar) { alumno in
            EditarAlumnoView(alumno: alumno)
        }
```

Qué esperas ver

Selecciona un alumno, pulsa Editar, cambia el apellido y pulsa Listo. La lista y el detalle se actualizan al instante, y el cambio sobrevive al reiniciar.

Qué pasó por dentro

@Bindable enlaza cada TextField directamente a la propiedad del objeto. Al escribir, el objeto cambia; SwiftData detecta el cambio y lo guarda solo. En Core Data teníamos que cambiar la propiedad y llamar save() nosotros.

Paso 10 · Buscar con .searchable

1. Agrega el estado de búsqueda y el arreglo filtrado:

```
    @State private var busqueda = ""                // Texto de búsqueda

    // Alumnos que coinciden con la búsqueda por apellido
    private var alumnosFiltrados: [Alumno] {
        if busqueda.isEmpty { return alumnos }
        return alumnos.filter { $0.apellido.localizedCaseInsensitiveContains(busqueda) }
    }
```

2. En la lista cambia ForEach(alumnos) por **ForEach(alumnosFiltrados)** y agrega, debajo de .onDelete:

```
            .searchable(text: $busqueda, prompt: "Buscar por apellido")
```

3. En deleteAlumnos cambia alumnos[index] por **alumnosFiltrados[index]**. ¿Por qué? Porque los índices que entrega .onDelete son de la lista que se ve en pantalla.

Qué esperas ver

Al deslizar hacia abajo la lista aparece una barra de búsqueda. Al escribir "gar" quedan solo los apellidos que contienen "gar".

Paso 11 · Relación de uno a muchos: Curso y Alumno

1. Crea el archivo **Curso.swift**:

Archivo: **Curso.swift**

```
import Foundation
import SwiftData

// DEMOSTRACIÓN: relación 1 a muchos (un Curso tiene muchos Alumnos)
@Model
class Curso {
    var nombre: String

    // deleteRule: qué pasa con los alumnos si se borra el curso
    //   .nullify -> los alumnos quedan sin curso
    //   .cascade -> también se borran los alumnos
    //   .deny    -> no deja borrar el curso mientras tenga alumnos
    @Relationship(deleteRule: .nullify, inverse: \Alumno.curso)
    var alumnos: [Alumno] = []

    init(nombre: String) {
        self.nombre = nombre
    }
}
```

2. En **Alumno.swift** agrega la propiedad del lado "uno":

```
    var curso: Curso?         // Relación: un alumno pertenece a un curso (opcional)
```

3. En **ModelContainerProvider.swift** cambia el esquema a **Schema([Alumno.self, Curso.self])**. También en el #Preview de ContentView: .modelContainer(for: [Alumno.self, Curso.self], inMemory: true).
4. En **ContentView.swift** agrega, en el detalle, la línea del curso y, en la barra, un botón de demostración con su función:

```
                    Text("Curso: \(alumno.curso?.nombre ?? "Sin curso")")
```

```
                ToolbarItem {
```

```
                    Button {
                        asignarCursoDemo()
                    } label: {
                        Label("Curso demo", systemImage: "book")
                    }
                }
```

```
    // DEMOSTRACIÓN de relación: crea un curso y se lo asigna a todos los alumnos
    private func asignarCursoDemo() {
        let curso = Curso(nombre: "Programación en Móviles")
        modelContext.insert(curso)
        for alumno in alumnos {
            alumno.curso = curso
        }
    }
```

Qué esperas ver

Al pulsar el libro, todos los alumnos pasan a mostrar "Curso: Programación en Móviles" en su detalle.

Ojo

Si la app se cierra al ejecutar con un error de migración, el archivo antiguo no coincide con el nuevo esquema. Borra la app del simulador (mantén pulsado el ícono ▸ Eliminar app) y ejecuta de nuevo. Es normal en desarrollo.

Qué pasó por dentro

@Relationship(deleteRule: .nullify, inverse: \Alumno.curso) declara el lado "muchos" y su inverso. Como en Core Data, basta asignar alumno.curso = curso. La regla **.nullify** deja a los alumnos sin curso si se borra el curso; **.cascade** los borraría y **.deny** impediría borrar el curso.

Commit y push: "Lab11 parte 2: editar, buscar y relación con SwiftData".

Parte 3 · Comparación y preguntas de comprensión

Responde en tu informe, con tus palabras (máximo tres líneas cada una):

1. ¿Qué diferencia hay entre **insertar** un objeto en el contexto y **guardar** el contexto? ¿En cuál de los dos frameworks tuviste que hacer ambas cosas a mano?
2. En UIKit llamamos reloadData(); en SwiftUI no. ¿Qué parte de SwiftData hace que la vista se actualice sola?
3. Si cambias el nombre de un atributo en el modelo y ya tenías datos guardados, ¿qué puede pasar? ¿Qué harías en desarrollo?
4. ¿Qué delete rule elegirías para la relación Grupo ▸ Contactos? ¿Y para Cliente ▸ Ventas? Justifica.
5. Un compañero dice que su app "guarda datos" pero al cerrarla desaparecen. ¿Qué dos cosas revisarías primero en cada framework?

VIII. Trabajo de la semana ()

Estas tareas **no se hacen en clase**: se desarrollan durante la semana y se entregan y defienden en la próxima sesión. Ambas apps tienen **inicio de sesión, registro y dos roles**, como una app de empresa.

Grupo	Caso y tecnología	Qué se evalúa
A	**Ventas** · UIKit + Core Data · 4 entidades y 9 pantallas · roles Vendedor y Supervisor	Login y roles, relaciones, stock, descuento por categoría, IGV, anulación con motivo y resumen del supervisor

Grupo	Caso y tecnología	Qué se evalúa
B	**Compras** · SwiftUI + SwiftData · 4 entidades y 9 pantallas · roles Comprador y Supervisor	Login y roles, relaciones, costo promedio, IGV como crédito fiscal, aprobación de compras grandes, cuentas por pagar y resumen

Reglas de trabajo: primero el expediente, después la IA

Pueden usar IA, pero **no como punto de partida**. La IA responde bien cuando uno sabe qué pedir; por eso se exige un expediente del trabajo antes de abrirla.

1. **Expediente (2 a 3 páginas, plantilla en el anexo):** entidades y atributos, relaciones y reglas de borrado, matriz de permisos por rol, reglas de negocio con ejemplos numéricos, pantallas dibujadas a mano o con la maqueta, casos de prueba y plan de commits.
2. **Visto bueno del docente:** muestren el expediente antes de usar IA (en clase o por el canal del curso). Sin visto bueno, no se pasa a la construcción asistida.
3. **Ramas en el repositorio:** main (expediente y entrega final), manual (lo que hicieron sin ayuda) e ia-asistida (lo que se hizo con IA).
4. **PROMPTS.md:** cada consulta a la IA se anota con el formato Contexto, Tarea, Restricciones, Formato esperado y qué hicieron con la respuesta (la aceptaron, la modificaron y por qué).
5. **Commit y push** al terminar cada pantalla, con mensajes claros. Un solo commit al final se considera trabajo no documentado.
6. **Defensa oral:** en la próxima clase, cada integrante explica una parte del código elegida por el docente, sin ayuda.

> **Ojo**
> Código que no pueden explicar, aunque funcione, no suma puntos en el criterio de defensa.

Inicio de sesión, registro y roles

- **Registro:** nombre completo, usuario, clave y rol. El usuario debe ser **único** (sin distinguir mayúsculas), la clave tiene mínimo 6 caracteres y debe coincidir con su confirmación. Al registrarse, la persona entra directo a la app.
- **Login:** usuario y clave. Si no coinciden, un mensaje genérico ("Usuario o clave incorrectos") sin decir cuál falló.
- **Sesión:** la app recuerda al usuario conectado mientras está abierta. La pestaña o el menú de **Cuenta** muestra su nombre y rol y tiene el botón **Cerrar sesión**, que vuelve al login.
- **Roles:** las pestañas y los botones cambian según el rol. Cada venta o compra guarda **quién la registró** (relación con Usuario).

> **Ojo**
> Por tratarse de una práctica, la clave se guarda como texto y el rol lo elige quien se registra. **Esto no es seguro en una app real:** en la próxima clase veremos cómo guardar un hash de la clave y por qué el rol no debe elegirse libremente.

Grupo A · Caso Ventas (UIKit + Core Data)

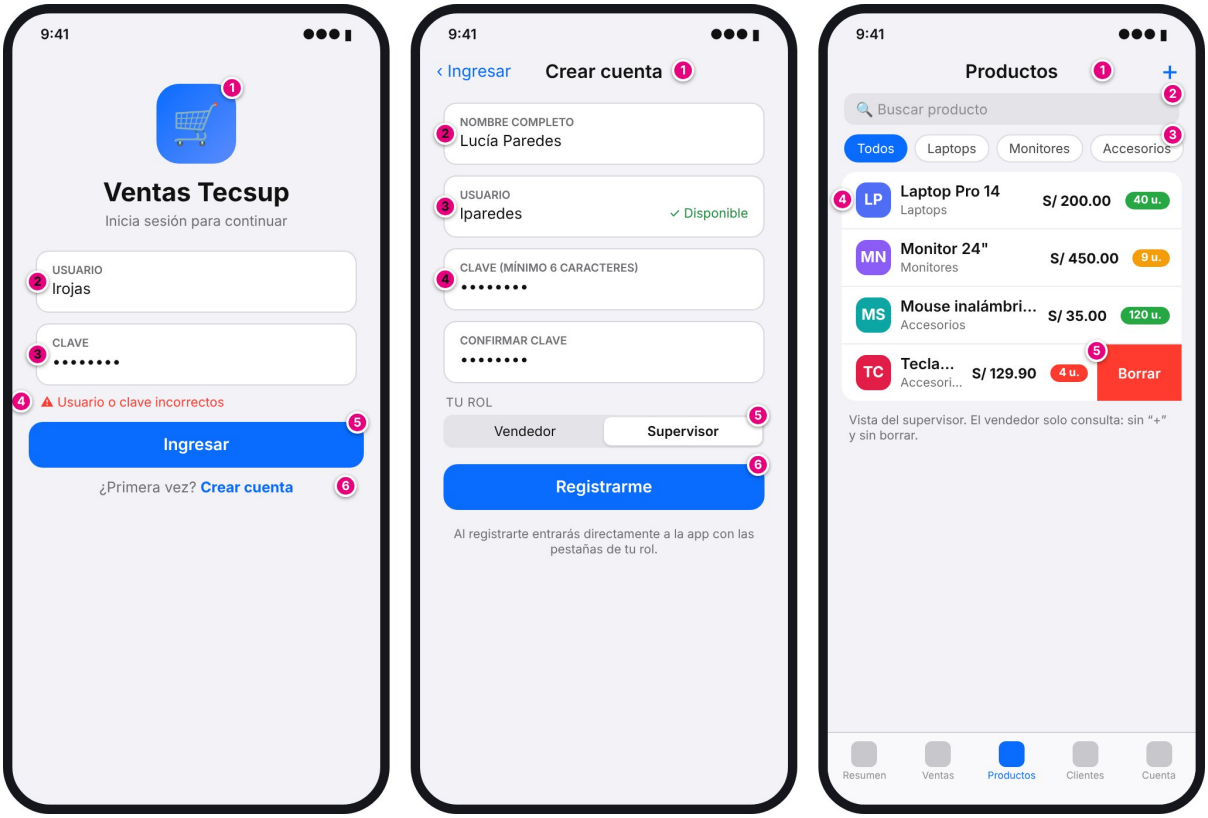
Una tienda registra productos, clientes y ventas. La app usa un **UITabBarController** cuyas pestañas dependen del rol.

Entidad	Atributos	Relaciones
Usuario	nombre, usuario, clave (String), rol (String: Vendedor o Supervisor)	ventas: To Many ▸ Venta (Deny)
Producto	nombre, categoria (String), precio (Double), stock (Int)	ventas: To Many ▸ Venta (Deny)

Entidad	Atributos	Relaciones
Cliente	nombre, documento (String)	ventas: To Many ▸ Venta (Deny)
Venta	fecha (Date), cantidad (Int), precioUnitario, descuento, igv, total (Double), anulada (Bool), motivoAnulacion (String, opcional)	vendedor: To One ▸ Usuario; producto: To One ▸ Producto; cliente: To One ▸ Cliente

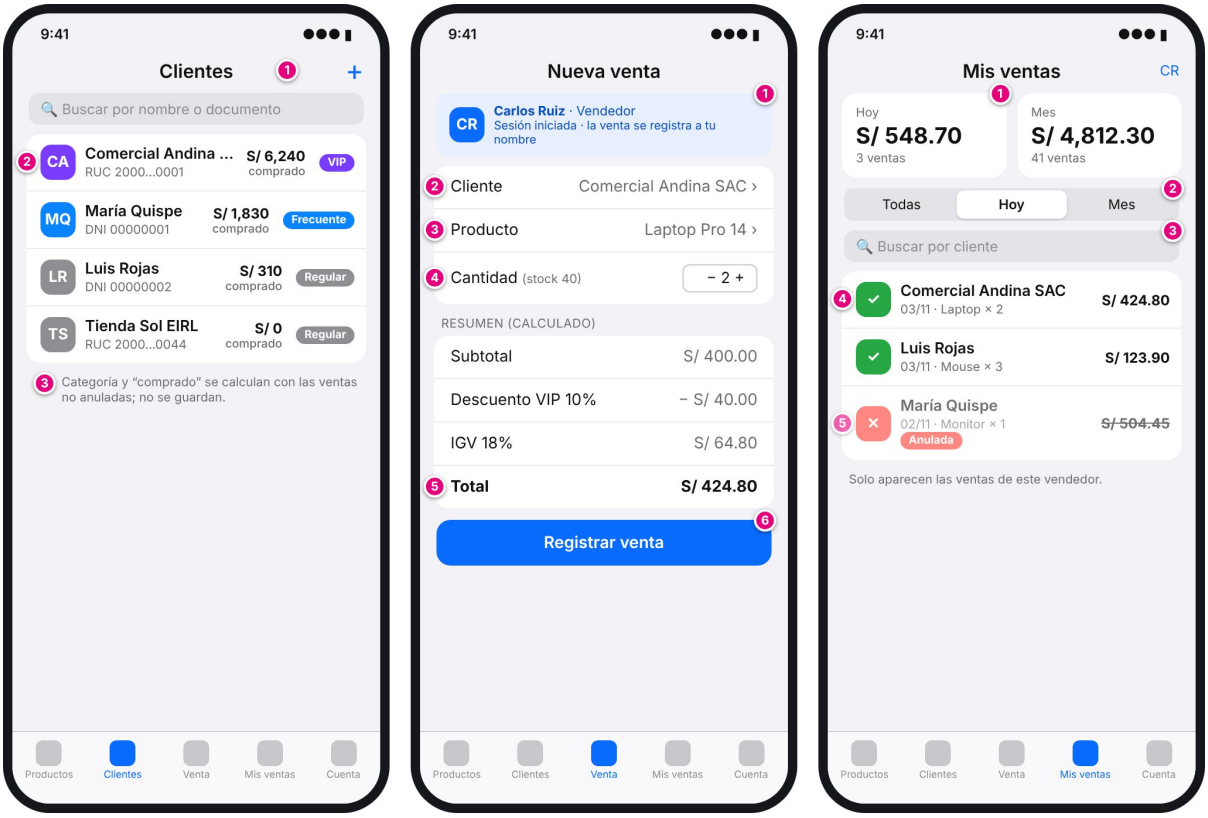
Qué puede hacer	Vendedor	Supervisor
Pestañas	Productos, Clientes, Venta, Mis ventas, Cuenta	Resumen, Ventas, Productos, Clientes, Cuenta
Productos	Solo consultar y buscar	Crear, editar y borrar
Clientes	Crear, editar y consultar	Crear, editar, consultar y borrar
Ventas	Registrar y ver **solo las suyas**	Ver **todas**, filtrar por vendedor y **anular** con motivo
Resumen	No lo ve	Panel con indicadores, gráfico y ranking

Pantallas que deben diseñar (9)

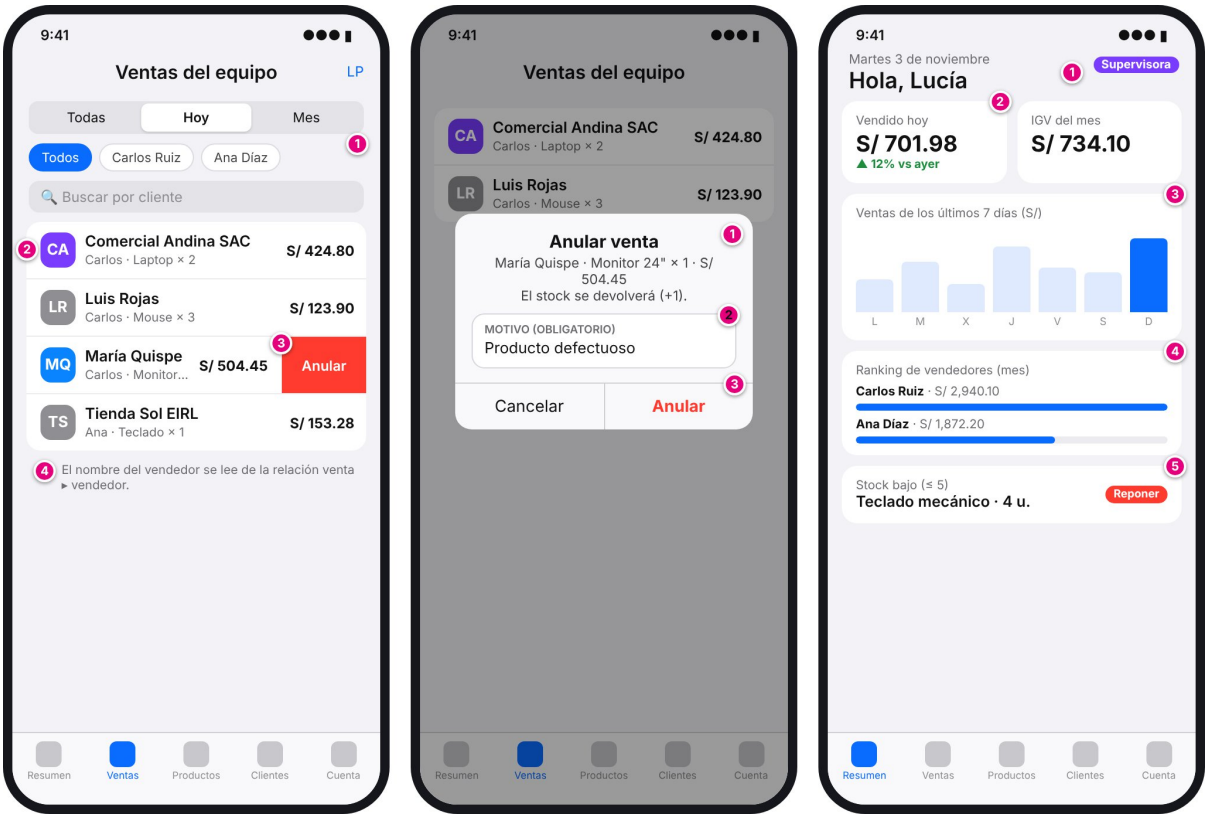


Pant.	Leyenda de la maqueta
A1	① Logo y nombre · ② Usuario · ③ Clave oculta · ④ Mensaje de error genérico · ⑤ Ingresar · ⑥ Ir a Crear cuenta
A2	① Título y volver al login · ② Nombre completo · ③ Usuario único, con aviso si ya existe · ④ Clave (mínimo 6) y su confirmación · ⑤ Rol: Vendedor o Supervisor · ⑥ Registrarme (entra directo a la app)

Pant.	Leyenda de la maqueta
A3	① Título y botón + (solo supervisor) · ② Búsqueda · ③ Chips de categoría (predicate) · ④ Fila con stock en color: verde más de 10, ámbar de 6 a 10, rojo 5 o menos · ⑤ Deslizar para borrar (solo supervisor; bloqueado si tiene ventas)



Pant.	Leyenda de la maqueta
A4	① Botón + · ② Fila con iniciales, nombre, documento y total comprado · ③ Categoría y total calculados con las ventas no anuladas
A5	① Vendedor de la sesión (automático, no editable) · ② Cliente · ③ Producto · ④ Cantidad con stepper y stock disponible · ⑤ Resumen calculado (subtotal, descuento, IGV, total) · ⑥ Registrar venta
A6	① Indicadores propios: hoy y mes · ② Segmented Todas / Hoy / Mes · ③ Búsqueda por cliente · ④ Venta propia · ⑤ Venta anulada (tachada, con etiqueta, sin botón)



Pant.	Leyenda de la maqueta
A7	① Chips por vendedor · ② Venta con el nombre del vendedor (relación venta ▸ vendedor) · ③ Deslizar para Anular (solo supervisor) · ④ Nota de la relación
A8	① Alerta con el resumen de la venta; avisa que el stock se devolverá · ② Motivo obligatorio · ③ Anular (deshabilitado mientras el motivo esté vacío)
A9	① Saludo con nombre y rol · ② Indicadores con comparación contra el día anterior · ③ Gráfico de barras de los últimos 7 días · ④ Ranking de vendedores del mes · ⑤ Productos con stock bajo

La pestaña **Cuenta** (Grupo A) es una pantalla sencilla: nombre, usuario, rol y el botón Cerrar sesión.

Reglas de negocio

- **Categoría del cliente (calculada, no guardada):** Regular si lo comprado (ventas no anuladas) es menor a S/ 1,000; Frecuente desde S/ 1,000; VIP desde S/ 5,000.
- **Descuento:** VIP 10 % del subtotal; los demás 0 %. **IGV:** 18 % sobre (subtotal − descuento). Total = subtotal − descuento + IGV.
- **Stock:** al registrar una venta baja el stock. Si la cantidad supera el stock, se muestra una alerta y no se registra.
- **Anular (solo supervisor):** exige un **motivo**, marca la venta como anulada, **devuelve** el stock y deja de contar en reportes y en la categoría del cliente.
- **Visibilidad:** el vendedor solo ve sus ventas (predicate por vendedor); el supervisor ve todas.
- **Borrado:** no se puede borrar un producto, cliente o usuario que tenga ventas (delete rule Deny); se informa con una alerta.

Niveles de entrega

Nivel	Contenido
Base (obligatorio)	Login, registro, sesión y pestañas por rol; las cuatro entidades con relaciones; venta con stock, descuento e IGV; mis ventas; ventas del equipo y anulación con

Nivel	Contenido
	motivo
Nivel +	Filtros por periodo y vendedor con NSPredicate (fechas y relación) y búsqueda por cliente
Reto	Panel del supervisor con gráfico de 7 días, ranking de vendedores y stock bajo, usando fetch requests

Casos de prueba (con resultado esperado)

N.º	Caso	Resultado esperado
1	Registrar a "lparedes" como Supervisor e intentar registrar otro "LPAREDES"	Se crea el primero y entra a la app; el segundo se rechaza por usuario repetido
2	Registrar con claves distintas o de 4 caracteres	Mensaje de error; no se crea el usuario
3	Login con clave incorrecta	Mensaje "Usuario o clave incorrectos"; no entra
4	Vendedor registra la venta de 2 Laptop a S/ 200.00 a un cliente VIP	Subtotal 400.00 · descuento 40.00 · IGV 64.80 · total 424.80 · stock 40 a 38
5	Vendedor registra 3 Mouse de S/ 35.00 a un cliente Regular	Subtotal 105.00 · IGV 18.90 · total 123.90
6	Venta de 50 unidades con stock 40	Alerta y no se registra; el stock no cambia
7	Un vendedor abre Mis ventas	Solo aparecen sus ventas y ninguna fila permite Anular
8	El supervisor anula la venta del caso 4 sin escribir motivo, y luego con motivo	Sin motivo no se puede; con motivo queda anulada, el stock vuelve a 40 y el resumen baja en 424.80
9	Borrar un cliente que tiene ventas	Se impide y se muestra un mensaje

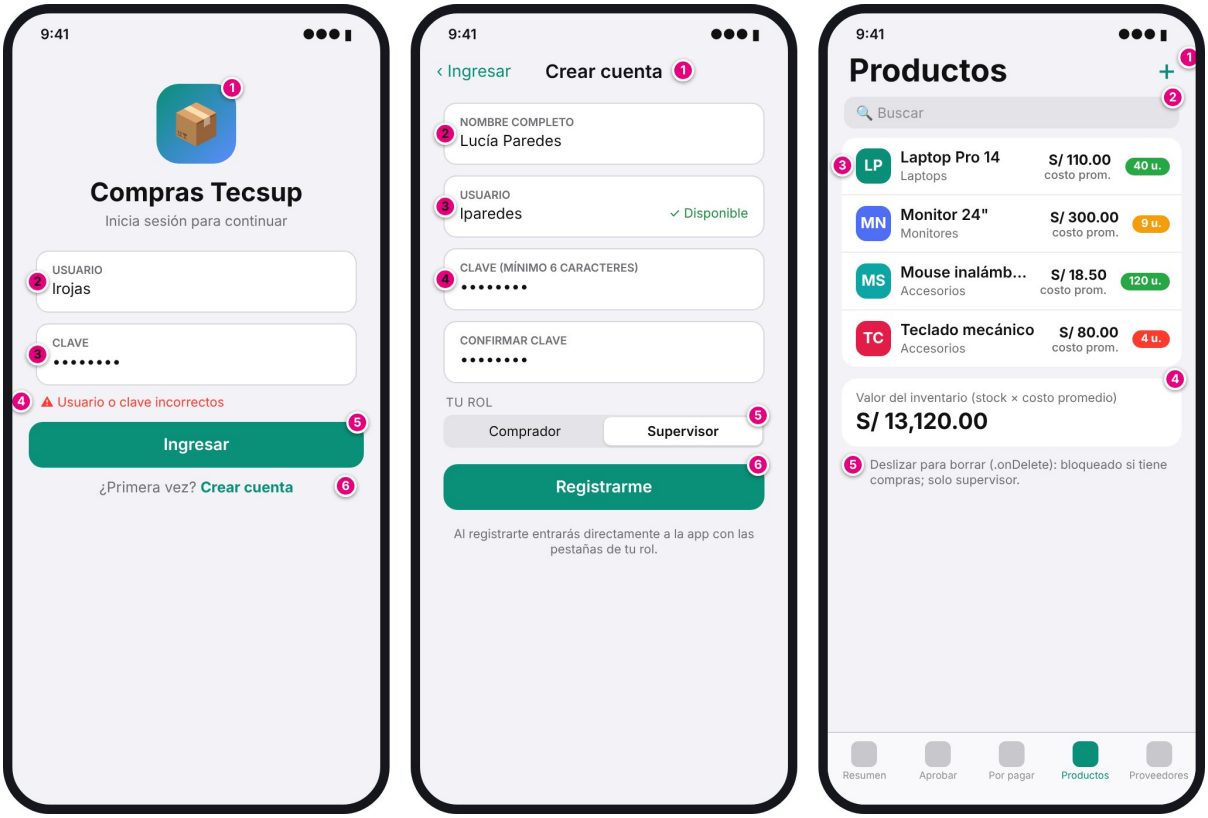
Grupo B · Caso Compras (SwiftUI + SwiftData)

Una distribuidora registra sus proveedores, el costo de sus productos y las compras al contado o al crédito. La app usa un **TabView** cuyas pestañas dependen del rol.

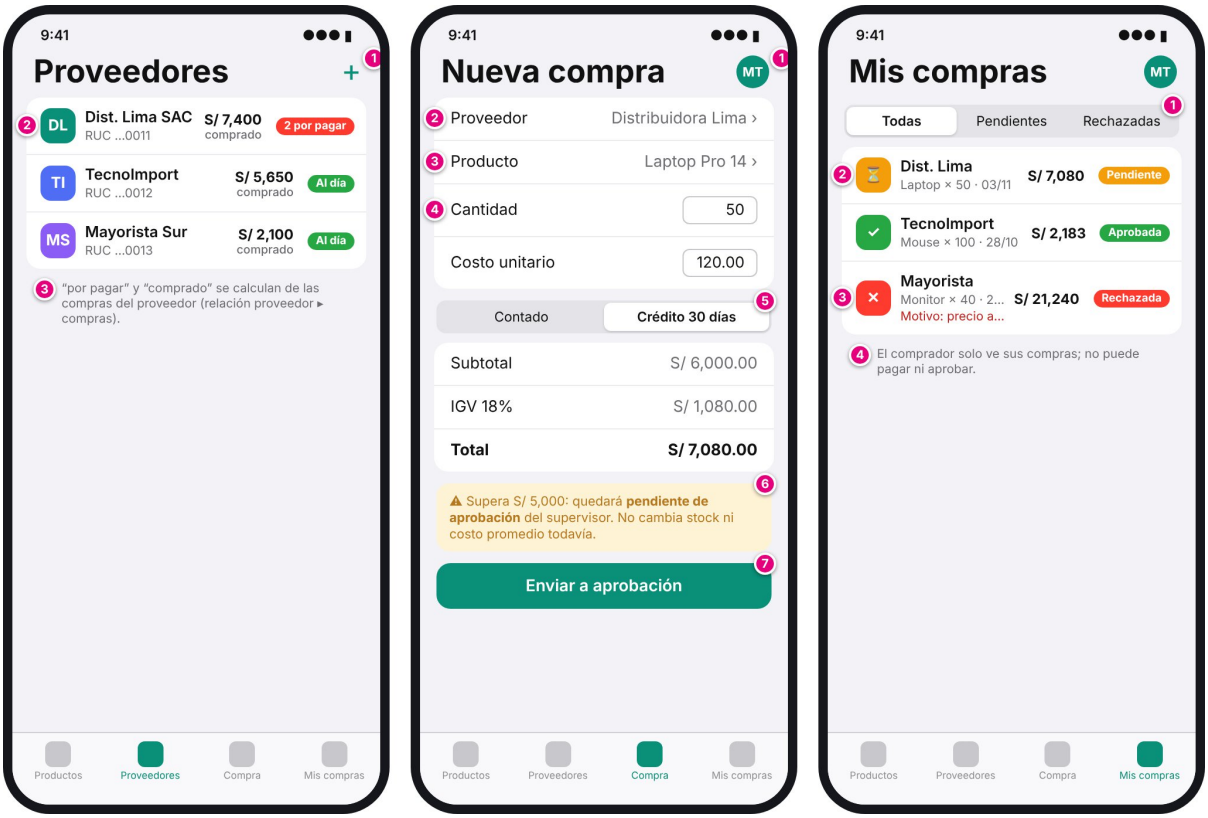
Clase @Model	Propiedades	Relaciones
Usuario	nombre, usuario, clave (String), rol (String: Comprador o Supervisor)	compras: [Compra] (.deny, inverse)
Proveedor	nombre, ruc (String)	compras: [Compra] (.deny, inverse)
Producto	nombre, categoria (String), stock (Int), costoPromedio (Double)	compras: [Compra] (.deny, inverse)
Compra	fecha (Date), cantidad (Int), costoUnitario, igv, total (Double), alCredito (Bool), vence (Date), estado (String: Pendiente, Aprobada o Rechazada), pagada (Bool), motivoRechazo (String, opcional)	comprador: Usuario?; proveedor: Proveedor?; producto: Producto?

Qué puede hacer	Comprador	Supervisor
Pestañas	Productos, Proveedores, Compra, Mis compras	Resumen, Aprobar, Por pagar, Productos, Proveedores
Productos y proveedores	Solo consultar y buscar	Crear, editar y borrar
Compras	Registrar y ver **solo las suyas**	Aprobar o rechazar, ver todo y **pagar**
Cuenta	Menú del avatar con Cerrar sesión	Menú del avatar con Cerrar sesión

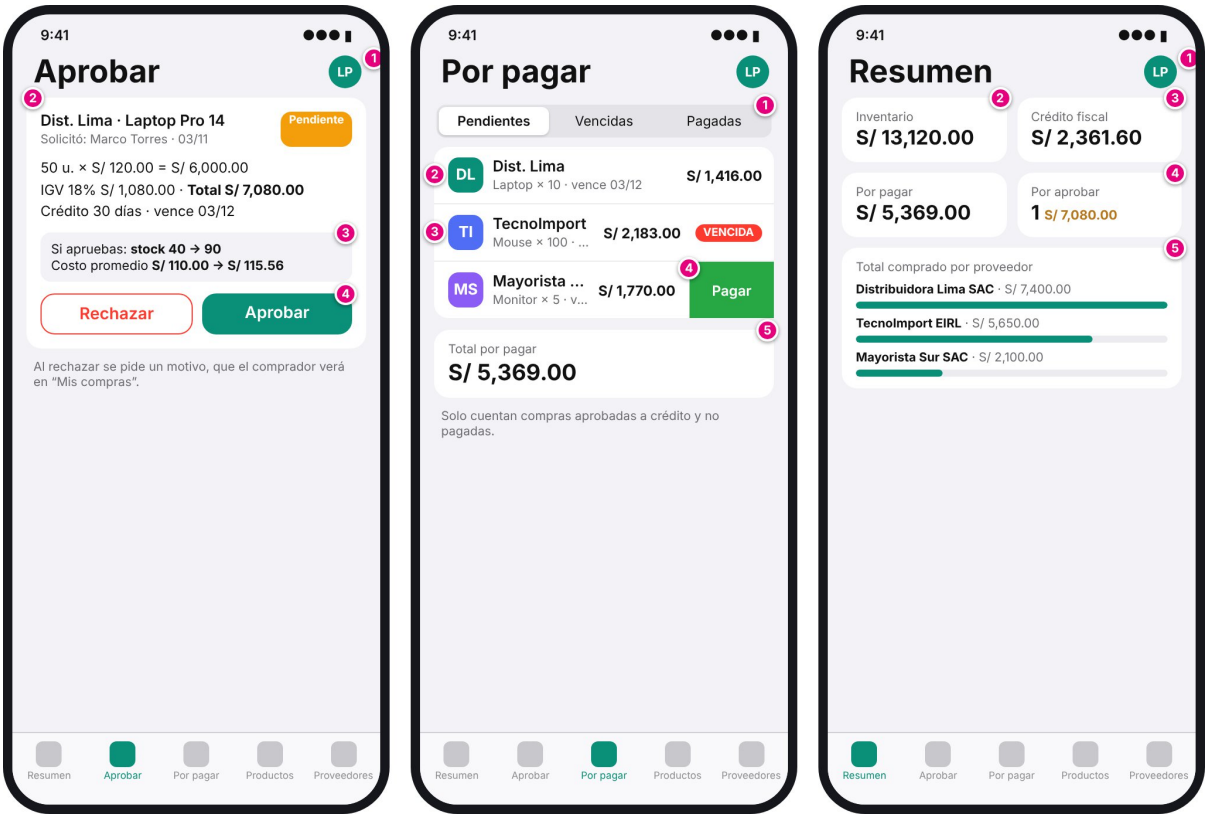
Pantallas que deben diseñar (9)



Pant.	Leyenda de la maqueta
B1	① Logo y nombre · ② Usuario · ③ Clave oculta · ④ Mensaje de error genérico · ⑤ Ingresar · ⑥ Ir a Crear cuenta
B2	① Título y volver al login · ② Nombre completo · ③ Usuario único, con aviso si ya existe · ④ Clave (mínimo 6) y su confirmación · ⑤ Rol: Comprador o Supervisor · ⑥ Registrarme (entra directo a la app)
B3	① Botón + (solo supervisor) · ② Búsqueda · ③ Fila con costo promedio y stock en color · ④ Valor del inventario (stock × costo promedio) · ⑤ Deslizar para borrar (.onDelete; solo supervisor y bloqueado si tiene compras)



Pant.	Leyenda de la maqueta
B4	① Botón + (solo supervisor) · ② Fila con insignia "por pagar" o "Al día" y total comprado · ③ Valores calculados desde las compras del proveedor
B5	① Avatar de la sesión (menú con Cerrar sesión) · ② Proveedor · ③ Producto · ④ Cantidad · ⑤ Contado o Crédito 30 días · ⑥ Aviso cuando el total llega a S/ 5,000 · ⑦ Enviar a aprobación (si es menor, el botón dice Registrar compra)
B6	① Segmented Todas / Pendientes / Rechazadas · ② Compra pendiente de aprobación · ③ Compra rechazada con su motivo · ④ El comprador solo ve las suyas



Pant.	Leyenda de la maqueta
B7	① Avatar de la sesión · ② Tarjeta de la compra pendiente con quién la solicitó · ③ Vista previa del efecto: stock y costo promedio antes y después · ④ Rechazar (pide motivo) o Aprobar
B8	① Segmented Pendientes / Vencidas / Pagadas · ② Compra a crédito pendiente · ③ Etiqueta VENCIDA · ④ Deslizar Pagar (solo supervisor) · ⑤ Total por pagar
B9	① Avatar de la sesión · ② Valor del inventario · ③ Crédito fiscal (IGV) · ④ Compras por aprobar (cantidad e importe) · ⑤ Total comprado por proveedor con barras

Reglas de negocio

- **IGV:** 18 % sobre el subtotal (cantidad × costo unitario). Es **crédito fiscal**: se acumula para el resumen. Total = subtotal + IGV.
- **Aprobación:** una compra con total **menor a S/ 5,000** queda Aprobada al instante. Desde **S/ 5,000** queda **Pendiente** y todavía no cambia stock, costo promedio ni cuentas por pagar. El supervisor la aprueba (se aplican los efectos) o la rechaza con un motivo (no cambia nada).
- **Costo promedio ponderado (al aprobar):** nuevo = (stock actual × costo promedio + cantidad × costo unitario) / (stock actual + cantidad).
- **Contado:** queda pagada. **Crédito 30 días:** queda pendiente de pago y vence a los 30 días. Pendiente de pago con fecha vencida se muestra como VENCIDA.
- **Pagar (solo supervisor):** marca la compra como pagada y sale de Pendientes.
- **Visibilidad y borrado:** el comprador solo ve sus compras. No se puede borrar un producto, proveedor o usuario con compras (.deny).

Niveles de entrega

Nivel	Contenido
Base (obligatorio)	Login, registro, sesión y pestañas por rol; las cuatro clases con relaciones; compra con IGV, contado o crédito y costo promedio; mis compras; aprobación y pago

Nivel	Contenido
Nivel +	Filtros Pendientes / Vencidas / Pagadas con #Predicate y búsqueda por proveedor
Reto	Resumen del supervisor con inventario, crédito fiscal, por pagar, por aprobar y total por proveedor

Casos de prueba (con resultado esperado)

N.º	Caso	Resultado esperado
1	Registrar a "mtorres" como Comprador e intentar repetir el usuario	Se crea el primero y entra a la app; el repetido se rechaza
2	Login con clave incorrecta	Mensaje "Usuario o clave incorrectos"; no entra
3	Producto con stock 40 y costo promedio 110.00. Compra de 10 a 120.00 al crédito	Subtotal 1,200.00 · IGV 216.00 · total 1,416.00 · queda Aprobada · stock 50 · costo promedio 112.00 · vence a 30 días
4	La misma compra, pero al contado	Queda pagada y no aparece en Pendientes de pago
5	Compra de 50 a 120.00 (total 7,080.00)	Queda Pendiente de aprobación; el stock sigue en 40 y no suma a por pagar
6	El supervisor aprueba la compra del caso 5	Stock 90 · costo promedio 115.56 · aparece en Por pagar
7	El supervisor rechaza otra compra grande sin motivo y luego con motivo	Sin motivo no se puede; con motivo queda Rechazada y el comprador ve el motivo; no cambia el stock
8	Compra a crédito con vencimiento ya pasado, y luego Pagar	Aparece en Vencidas con VENCIDA; al pagar pasa a Pagadas y el total por pagar baja
9	Un comprador abre Mis compras e intenta pagar	Solo ve sus compras y no tiene la opción de Pagar

Ícono de la app (ambos grupos)

La app debe tener un **ícono profesional** visible en la pantalla de inicio del simulador, y su nombre corto debajo ("Ventas" o "Compras").



Ícono de Ventas (Grupo A)



Ícono de Compras (Grupo B)



Cómo se verá en la pantalla de inicio

1. Usa el ícono de 1024 × 1024 px que se entrega como ejemplo, o diseña uno propio (Figma, Canva, SF Symbols sobre un fondo de color).
2. En Xcode abre **Assets.xcassets ▸ AppIcon** y arrastra la imagen al recuadro de 1024 pt.
3. Requisitos: PNG cuadrado, **sin transparencia y sin esquinas redondeadas** (iOS las redondea solo), con un símbolo simple y buen contraste.
4. Nombre bajo el ícono: target ▸ General ▸ **Display Name** = Ventas o Compras.
5. Para verlo: ejecuta la app, detenla y pulsa el botón Inicio del simulador (Shift + Cmd + H). Toma una captura y agrégala a la entrega.

Pistas para la sesión (sin la solución)

La demostración de clase no incluye login. Estas son las piezas que necesitan; armarlas es parte del trabajo.

Pieza	UIKit + Core Data (Grupo A)	SwiftUI + SwiftData (Grupo B)
Usuario único	Fetch con NSPredicate "usuario ==[c] %@" y comprobar si devuelve resultados	FetchDescriptor con #Predicate (guarda el texto en un let antes de usarlo) y fetchCount
Sesión	Una clase con un único objeto compartido (static let) y una variable usuario actual	Una clase @Observable con el usuario actual, puesta con .environment(...) en la App
Pantalla inicial	Mostrar el login y, al entrar, abrir un UITabBarController con las pestañas del rol	if let usuario = sesion.usuario { TabView(...) } else { LoginView() }
Ver solo lo propio	Predicate "vendedor == %@" con el usuario de la sesión	@Query con #Predicate sobre el comprador, o filtrar el arreglo de @Query
Cerrar sesión	Poner la variable en nil y volver al login	Poner la variable en nil: la vista cambia sola

Cierre conjunto de A y B (en la próxima clase)

Con los datos de ambas apps armamos el cierre de IGV del mes: **IGV por pagar = IGV de ventas − crédito fiscal de compras**. Por ejemplo, con IGV de ventas S/ 734.10 y crédito fiscal S/ 2,361.60 resulta un saldo a favor de S/ 1,627.50. Cada grupo trae sus cifras para completar este cuadro en el aula.

Cronograma sugerido de la semana

Día	Entregable
Días 1 y 2	Expediente completo (con matriz de permisos) y visto bueno del docente
Días 3 y 4	Modelo de datos, registro, login, sesión y pestañas por rol, con commit y push
Días 5 y 6	Resto de pantallas y reglas, casos de prueba, ícono, captura en el simulador y PROMPTS.md final
Próxima clase	Entrega en GitHub, demostración, cierre conjunto y defensa oral

Rúbrica (20 puntos)

Criterio	Puntos	Evidencia
Expediente previo con visto bueno	3	Documento, matriz de permisos y fecha del visto bueno

Criterio	Puntos	Evidencia
Modelo de datos y relaciones correctas	3	Entidades, relaciones y delete rules
Registro, login, sesión y roles	4	Los casos de usuario y la visibilidad por rol
Reglas de negocio y casos de prueba	4	Los nueve casos con el resultado esperado
Pantallas fieles a la maqueta y ícono	2	Capturas y pantalla de inicio
Repositorio, commits y PROMPTS.md	1	Historial de ramas y consultas anotadas
Defensa oral individual	3	Explicación sin ayuda de una parte elegida

IX. Conclusiones

Redacta tres conclusiones propias sobre lo aprendido (puedes apoyarte en las preguntas de la Parte 3).

1. ……
2. ……
3. ……

Anexo · Plantilla del expediente del trabajo

Complétalo **antes** de usar IA. Dos a tres páginas bastan.

Sección	Qué debe contener
1. Integrantes y caso	Nombres, grupo (A o B) y nombre de la app
2. Modelo de datos	Cada entidad con sus atributos y tipos; relaciones y su delete rule, con un dibujo de las cajas y flechas
3. Roles y permisos	Tabla de qué ve y qué puede hacer cada rol, y qué pestañas tiene
4. Reglas de negocio	Cada regla con un ejemplo numérico resuelto a mano (descuento, IGV, stock, costo promedio o aprobación)
5. Pantallas	Las nueve pantallas con su propósito y qué dato lee o escribe cada una
6. Casos de prueba	Los nueve casos con su resultado esperado, más dos que ustedes inventen
7. Plan de commits	Qué pantalla o regla se entrega en cada commit
8. Plan de uso de IA	Qué tareas pedirán a la IA y cuáles harán a mano; cómo verificarán sus respuestas
9. Visto bueno	Fecha y firma o confirmación del docente